

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load the Fashion-MNIST dataset (60,000 training images and 10,000 testing images).
2. Display sample images from the dataset.
3. Flatten each 28×28 image into a 784-dimensional feature vector.
4. Normalize pixel values to the range $[0, 1]$.
5. Convert class labels into one-hot encoded vectors of size 10.

Results:

- Shape of Training data: (60000, 784)
- Shape of Testing data: (10000, 784)
- Shape of Training labels: (60000, 10)
- Shape of Testing labels: (10000, 10)

Inference: We have successfully transformed the MNIST Fashion dataset into normalised feature vectors using some pre processing techniques. Using One-Hot encoding we have transformed the class labels into completely formatted labels with a softmax output layer.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the Fashion-MNIST dataset.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Results:

- Training images: (60000, 28, 28)
- Testing images: (10000, 28, 28)
- Training labels: (60000,)
- Testing labels: (10000,)
- The first ten sample images are displayed.
- This is a balanced dataset as it has 6000 images in each of the 10 classes, we can infer this from the class distribution diagram.

Inference: The Fashion-MNIST dataset contains a balanced distribution of 10 clothing categories with equal samples per class. The sample images confirm the dataset is suitable for multi-class image classification.

Task 2: Data Preprocessing

- Flatten images from 28×28 pixels to 784-dimensional feature vectors.
- Normalize pixel values from the range $[0, 255]$ to $[0, 1]$.
- Print tensor shapes before and after preprocessing.

Results:

- Training images: (60000, 28, 28) $\rightarrow$ (60000, 784)
- Testing images: (10000, 28, 28) $\rightarrow$ (10000, 784)
- Pixel values are normalized to the range [0,1].

Inference: By flattening the images we converted them into a 1-d feature vector which are suitable for MLP training. By normalizing the images to the range of [0,1] , the image's pixel values are consistent which make the training stable and good convergence.

Task 3: Model Construction

Construct an MLP with

- Input layer with 784 neurons.
- Hidden Layer 1: Dense (128 neurons, ReLU activation).
- Hidden Layer 2: Dense (64 neurons, ReLU activation).
- Output Layer: Dense (10 neurons, Softmax activation).

Results:

- Input Shape: (*None*, 784)
- Total Parameters: 109,386
- Trainable Parameters: 109,386
- Non-trainable Parameters: 0
- The complete model architecture was verified using `model.summary()`.

Inference: We constructed the MLP using two hidden layers with ReLU activation and a softmax output layer which helps multi class classification.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss Function: Categorical Cross-Entropy
- Evaluation Metric: Accuracy
- Number of Epochs: 20
- Batch Size: 32

Results:

- Final Training Accuracy: 93.36%
- Final Validation Accuracy: 89.17%
- Final Training Loss: 0.1735
- Final Validation Loss: 0.3509

Inference: The MLP model with Adam optimization converged after 20 epochs. We ensured that the training and validation indicate good learning instead of overfitting the dataset.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Results:

- Accuracy: 88.60%
- Precision: 88.90%
- Recall: 88.60%
- F1-score: 88.66%

- Confusion matrix generated to visualize class-wise predictions.
- Classification report was generated for all the 10 classes containing precision, recall, F1-score.

Inference: The MLP model we trained for the given Fashion MNIST dataset achieved 88.60

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Randomized Search using **3-fold cross-validation (CV = 3)**.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

Results:

- Cross-validation folds used: **3**
- Hyperparameter tuning method: Randomized Search
- Best Cross-validation Accuracy: **88.28%**
- Best Hyperparameters:
 - Optimizer: SGD
 - Learning Rate: 0.1
 - Hidden Layers: 1
 - Neurons: 128
 - Activation Function: Tanh
 - Dropout Rate: 0.0
 - Batch Size: 32
 - Epochs: 30
- Optimized Test Accuracy: **88.92%**

Inference: We have only used 3 cross validation folds, the randomized search with these 3 cross validation folds gave the best hyperparameter value. Now the model is trained again with the best values. And hence an optimized model is achieved. We also observe that optimized model has a slight increase in the accuracy.

8. Mandatory Plots

1. Sample Images

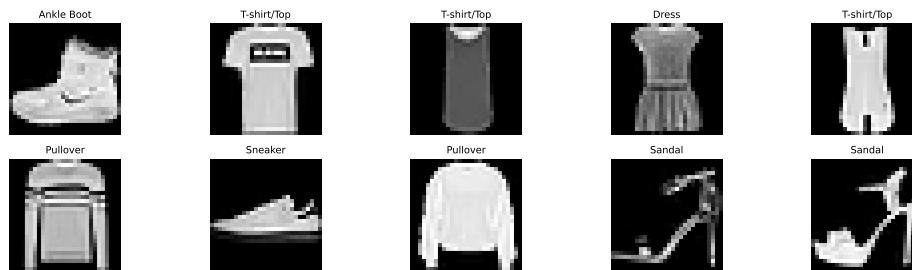


Figure 1: Sample Fashion-MNIST Images

Inference:

By displaying the sample images we can see how the grayscale is used for the dataset. We can also observe that there are 10 different classes like Boots, Shoes, Shirts, trousers, bags.

2. Class Distribution

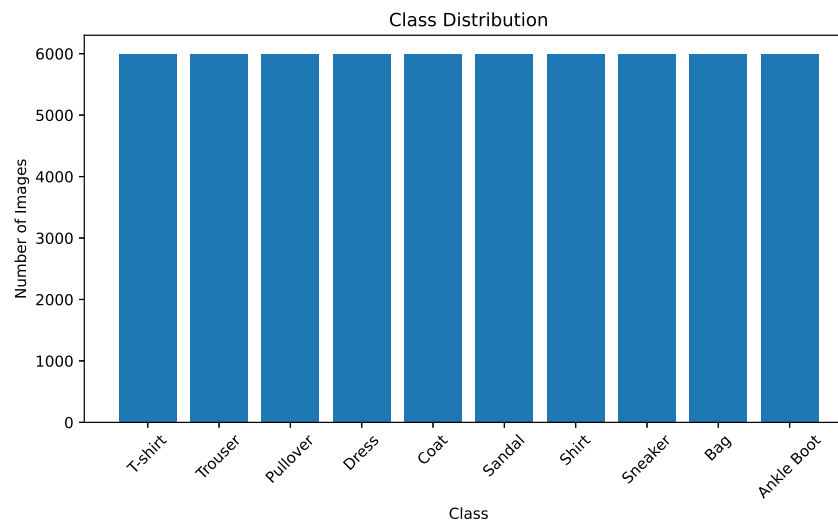


Figure 2: Class Distribution of Fashion-MNIST Dataset

Inference: This distribution plot shows that the dataset is balanced having 6000 samples/training images for each class. Since the dataset is balanced there will be no bias during the training.

3. Training Accuracy vs Epoch

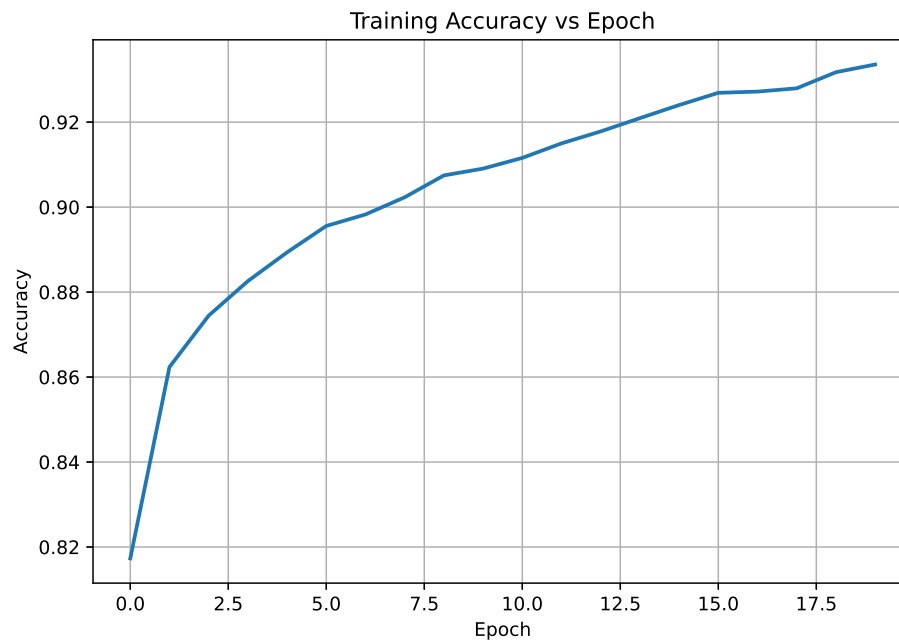


Figure 3: Training Accuracy vs Epoch

Inference:

We see that as we go through the epoch, we see that the accuracy is increasing as we increasing the number of epochs. The steady increase in the accuracy shows that the model has learned meaningful features from the training dataset.

4. Validation Accuracy vs Epoch

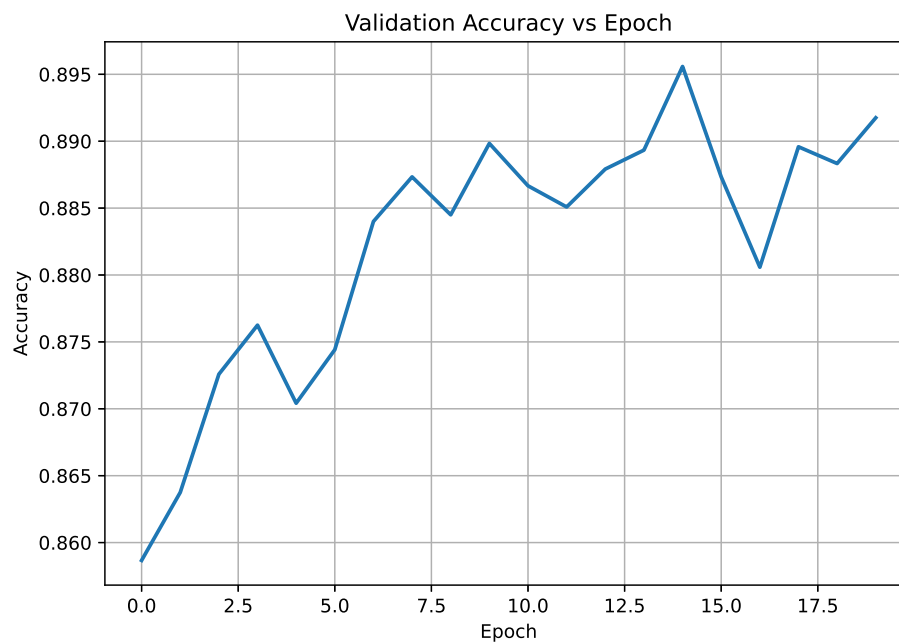


Figure 4: Validation Accuracy vs Epoch

Inference: Same as the training accuracy we see that the validation accuracy also increased in a stable manner, the training accuracy is reached approximately to 93

5. Training Loss vs Epoch

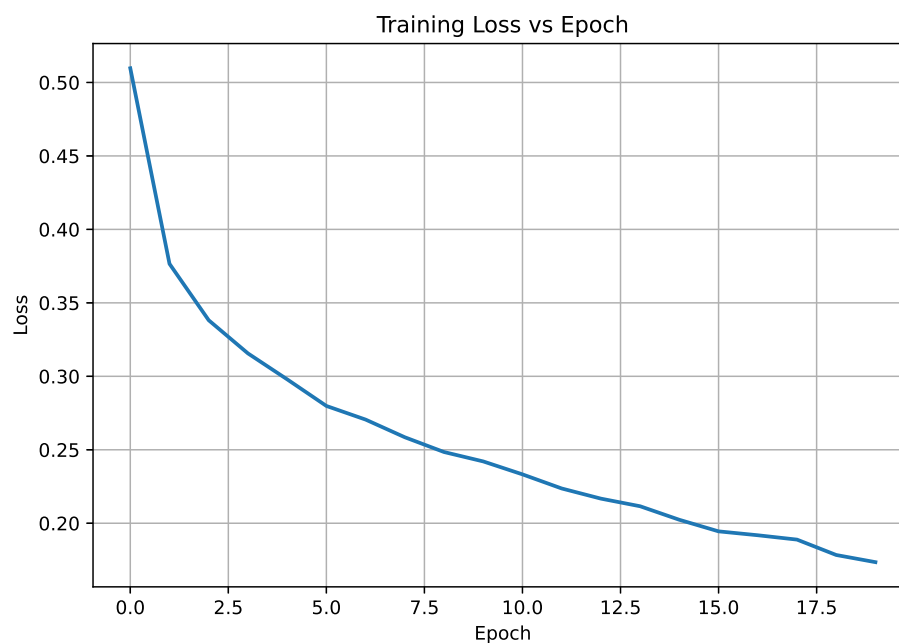


Figure 5: Training Loss vs Epoch

Inference:

As we increase the number of epochs we observe that the training loss is decreasing continuously during the training, the training loss had reached the value of 0.1735. So we can understand that the model came to near convergence.

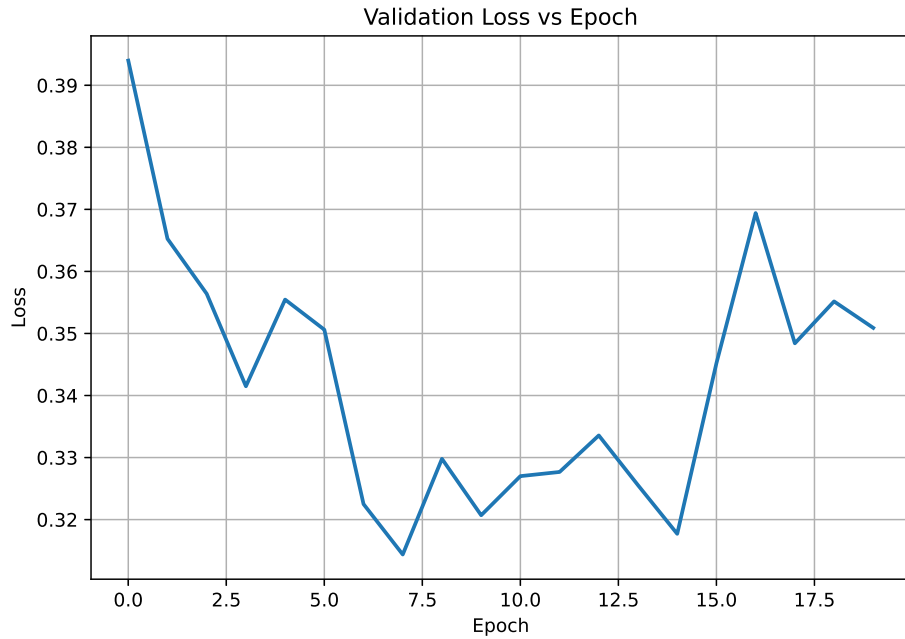
6. Validation Loss vs Epoch

Figure 6: Validation Loss vs Epoch

Inference:

Similarly as the training loss the validation loss also decreased over the epochs, It reached a final value of at around **0.3509**.

7. Confusion Matrix

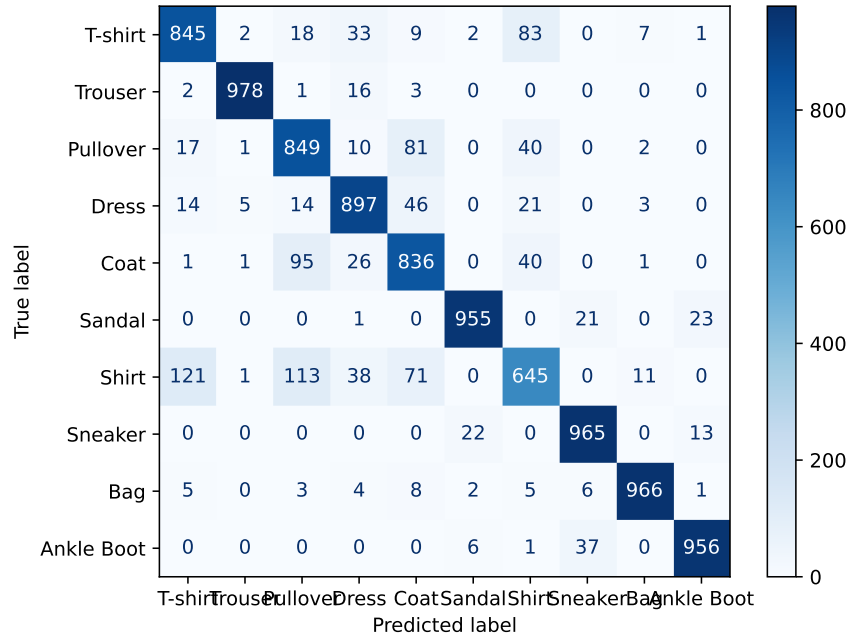


Figure 7: Confusion Matrix

Inference:

From the plotted confusion matrix we can observe that most of the images are classified correctly and there are minimal errors, this shows that the MLP model has trained well in an effective way.

8. Hyperparameter Search Results

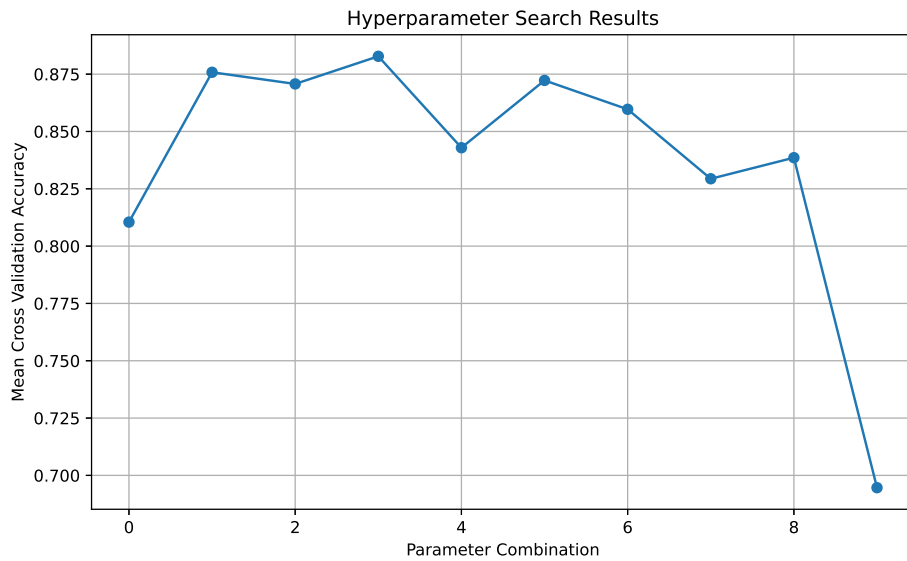


Figure 8: Hyperparameter Search Results

Inference:

We used 3 folds cross validation to tune the hyper parameters. The best parameters are selected and the model is retrained with those parameters, and this improved the performance of the MLP model compared to the baseline implementation.

9. Best Model Accuracy Comparison

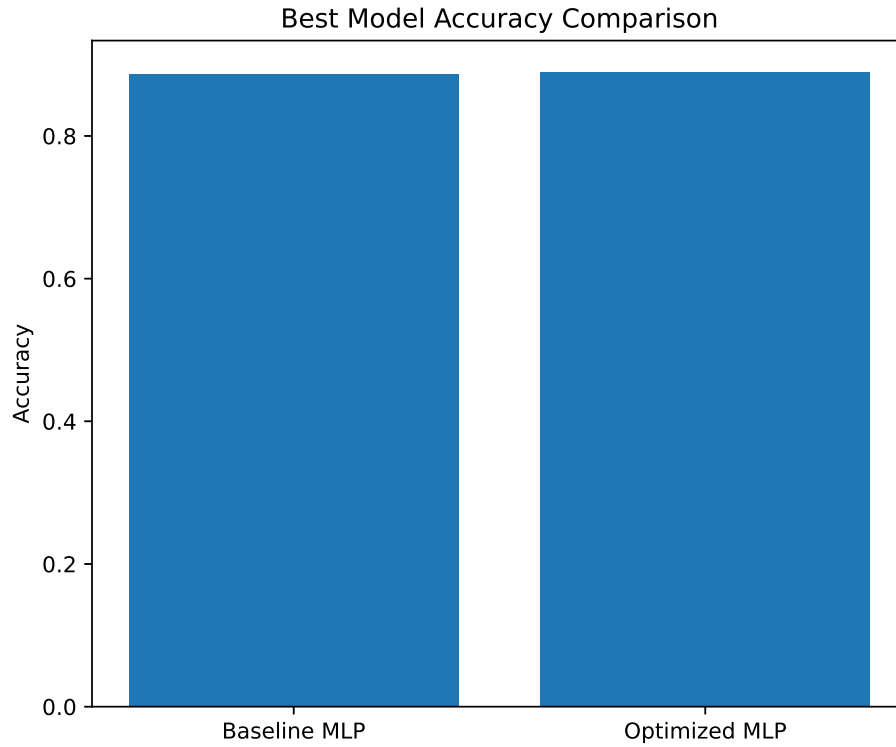


Figure 9: Baseline vs Optimized Model Accuracy

Inference: The model with tuned best parameters achieved the accuracy of **88.92%** which is slightly greater than the baseline models accuracy.(The baseline models accuracy is **88.60%**). With this we understand the importance of tuning the parameters.

9. Results

Best Hyperparameters

Parameter	Best Value
Hidden Layers	1
Hidden Neurons	128
Learning Rate	0.1
Batch Size	32
Optimizer	SGD
Activation Function	Tanh
Epochs	30
Dropout	0.0
Cross-validation Accuracy	88.28%
Testing Accuracy	88.92%

Performance Comparison

Metric	Baseline	Optimized
Accuracy	88.60%	88.92%
Precision	88.90%	88.92%
Recall	88.60%	88.92%
F1-score	88.66%	88.92%
Training Time	20 Epochs	30 Epochs

10. Discussion

1. Which optimization method was used?

We used 3 fold cross validation with Randomized Search to tune and find the best hyperparameters for the optimized version of MLP model.

2. Which hyperparameters were selected?

The best hyperparameters obtained and selected are: 1 hidden layer, 128 neurons, learning rate of 0.1, batch size of 32, SGD optimizer, Tanh activation function, 30 epochs, and no dropout.

3. Did optimization improve performance?

Yes, the optimized model increased the performance , we observe a slight increase in accuracy from 88.60% to 88.92%

4. Which hyperparameter had the greatest impact?

The learning rate and the optimizer has the impact in optimizing the model for best hyper parameters. Choosing the right values help the model to learn effectively without overfitting , the model cann be generalized well.

5. Compare Grid Search and Randomized Search.

Grid Search checks every possible combination of hyperparameters, which takes more time. Randomized Search tests only a selected number of combinations, making it faster while still giving good results. In this experiment, Randomized Search was used because it reduced the computation time.

6. Recommend the best MLP configuration.

When we observe the performed experiment , the best MLP model is the model with 1 hidden layer with 128 neurons, tanh activation function, SGD optimizer, 0.1 learning rate, 32 batch size, epochs - 30, and with no dropout. The model with this parameters achieved higher test accuracies.

11. Conclusion

In this experiment, a Multi-Layer Perceptron (MLP) model was implemented and trained on the Fashion-MNIST dataset for image classification. The dataset was preprocessed by flattening the images, normalizing the pixel values, and converting the labels into one-hot encoded vectors.

A baseline model was first developed and later improved using Randomized Search with 3-fold cross-validation for hyperparameter tuning. The optimized model achieved a testing accuracy of **88.92%**, which was slightly better than the baseline accuracy of **88.60%**. The experiment showed that selecting suitable hyperparameters can improve the performance of an MLP model. Overall, the objectives of the experiment were successfully achieved.

12. Source Code

The complete source code for this experiment was implemented using Python in Google Colaboratory. The notebook includes data preprocessing, model construction, training, evaluation, hyperparameter optimization, and visualization of the experimental results.

The complete source code is available at the following Google Drive link:

https://colab.research.google.com/drive/1zKusP9yNy_mvgs3mQvNP9NX5yre9AlwJ?usp=sharing

Git repository: https://github.com/LaasyaPriya-1238/DL_LAB.git

13. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.